```python
#Import Required Libraries

import pandas as pd # for work eith ata table
import numpy as np # for numbers and math
import warnings # To Control Warning Message
warnings.filterwarnings('ignore')   # Hide unimportant warnings

#Load Dataset

df_clean = pd.read_csv("C:\\Users\\OM-159\\Downloads\\
uber_rapido_driver_dirty.csv")
df = df.copy() # copy dataset for performing Oprations

print(f"Dataset loaded!")
print(f"Rows : {df.shape[0]}")
print(f"Columns : {df.shape[1]}")
```

```
Dataset loaded!
Rows : 50000
Columns : 33
```

```python
# See First 10 rows using head()
df.head(10)
```

```
   driver_id       driver_name  age       city        phone  \
0  DRV016649    Naresh Chauhan  NaN    Chennai   9255500096
1  DRV032785        Amit Gupta   45  Ahmedabad   9940345570
2  DRV011323     Yogesh Nair     53    Lucknow   9170487337
3  DRV022168       Manoj Patel   46  Hyderabad   9246425981
4  DRV040782  Vikram Chaudhary   25    KOLKATA      9282240
5  DRV003740  Sanjay Chaudhary  -32       Pune  -9120959055
6  DRV037372  Rakesh Chaudhary   21    Lucknow  -9109647964
7  DRV042083     Naveen Mishra  999      Delhi      9988246
8  DRV030189      Hitesh Singh   43    Kolkata   9994609866
9  DRV029154    Harish Chauhan   39    Chennai   9712392703

                         email        platform vehicle_type
vehicle_condition  \
0  naresh.chauhan@hotmail.com            uber         auto
good
1   amit.gupta@rediffmail.com            Uber         Mini
Excellent
2                          NaN            uber          SUV
avg
3     manoj.patel@hotmail.com          RAPIDO         Mini
NaN
4  vikram.chaudhary@gmail.com            Uber          suv
GOOD
5  sanjay.chaudhary@yahoo.com  Uber & Rapido         BIKE
excellent
6  rakesh.chaudhary@gmail.com  Uber & Rapido         BIKE
```

```
Average
7   naveen.mishra@hotmail.com          Uber           suv
Excellent
8                          NaN           UBER         Mini
Excellent
9    harish.chauhan@gmail.com           UBER         Bike
GOOD

  experience_level  ... driving_test_score_10 route_knowledge_score_10
\
0           0-1 yr  ...                   4.3                       5.8

1          Fresher  ...                   9.3                       NaN

2           0-1 yr  ...                   6.3                       9.0

3          1-3 yrs  ...                   8.7                       9.6

4          Fresher  ...                  12.7                       6.5

5          Fresher  ...                   8.8                       8.6

6           0-1 yr  ...                   8.1                       8.7

7           0-1 yr  ...                   8.8                       6.4

8           5+ yrs  ...                  -1.1                       7.9

9           0-1 yr  ...                  11.6                       7.7


  customer_handling_score_10 overall_interview_score_10  \
0                        7.7                        5.3
1                        9.0                        8.1
2                        5.5                        6.9
3                        5.3                        7.8
4                        7.9                        6.0
5                       10.5                       -1.4
6                        9.7                        8.2
7                        NaN                        6.0
8                       10.7                        8.3
9                        5.9                        6.7

   document_verification  background_check_status  interview_result  \
0                   Fail                      NaN           On Hold
1                    NaN                    Clear           On Hold
2                    NaN                    clear          Selected
3                unknown                    Clear          Selected
4                   FAIL                    clear          Rejected
5                   FAIL                      NaN          Selected
```

```
6                    NaN                  pending          Selected
7                    NaN                    Clear          Selected
8                   FAIL                      NaN          Selected
9                   FAIL                  Flagged           On Hold

     rejection_reason       onboarding_date  is_currently_active
0                 NaN                    -                FALSE
1                 NaN                    -                   No
2                 NaN          19-Apr-21                  FALSE
3                 NaN          5/27/2021                      0
4  Low rating history                    -                   NO
5                 NaN              Apr-23                     N
6                 NaN           8/8/2023                   TRUE
7                 NaN           2/6/2022                      0
8                 NaN  2022-06-08T00:00:00Z                   1
9                 NaN                    -                   N

[10 rows x 33 columns]
```

```
# last 10 rows

df.tail(10)
```

```
       driver_id      driver_name age       city       phone  \
49990  DRV001745    Bharat Sharma  35      Delhi         NaN
49991        NaN     Naresh Singh  49  Hyderabad  9082276515
49992  DRV028695     Bharat Dubey  40  Hyderabad  9724772991
49993  DRV026059     Vikram Dubey  48    Kolkata  9915689598
49994  DRV024759      AMIT PANDEY  42  Ahmedabad     9170227
49995  DRV046005      Sunil Dubey  26       Pune  9644914618
49996  DRV034162    Pankaj Shukla  38    Chennai  9204181688
49997  DRV026085     Vijay Mishra  30      Delhi  9289072455
49998  DRV025771  Ajay Srivastava  41    CHENNAI  9071408289
49999  DRV025843     Dinesh Reddy  24     Mumbai  9413636096

                            email platform vehicle_type
vehicle_condition  \
49990    bharat.sharma@hotmail.com      ola          NaN
good
49991     naresh.singh@hotmail.com   rapido         Auto
excellent
49992  bharat.dubey@rediffmail.com     uber         none
Poor
49993     vikram.dubey@hotmail.com    Uber         Mini
avg
49994   amit.pandey@rediffmail.com     UBER          NaN
NaN
49995       sunil.dubey@@yahoo.com   Rapido         Mini
NaN
49996      pankaj.shukla@gmail.com     UBER        Sedan
```

```
none
49997     vijay.mishra@hotmail.com     uber         Sedan
Average
49998  ajay.srivastava@hotmail.com     Uber         sedan
NaN
49999     dinesh.reddy@hotmail.com      ola          Bike
good

      experience_level  ... driving_test_score_10
route_knowledge_score_10  \
49990           0-1 yr  ...                   5.9
6.9
49991          1-3 yrs  ...                   5.0
5.8
49992          Fresher  ...                   7.1
9.9
49993          1-3 yrs  ...                  12.5
8.9
49994           5+ yrs  ...                   5.3
4.7
49995               na  ...                   6.7
5.9
49996          1-3 yrs  ...                  12.1
-1.2
49997          3-5 yrs  ...                   6.6
9.8
49998          Fresher  ...                   5.3
7.1
49999          Fresher  ...                   5.6
10.5

      customer_handling_score_10 overall_interview_score_10  \
49990                       -0.4                         7.7
49991                        NaN                         NaN
49992                        6.0                         7.2
49993                        8.1                         7.1
49994                       11.4                         5.7
49995                        8.3                         7.6
49996                        5.0                         7.0
49997                        8.2                         8.3
49998                        5.1                         6.1
49999                        3.9                         5.3

       document_verification  background_check_status
interview_result  \
49990                    NaN                  FLAGGED
NaN
49991                   Pass                  pending
Selected
49992                   Fail                    clear
```

```
Selected
49993                  Pending                    Pending
Waitlist
49994                      NaN                    Pending
Rejected
49995                  Unknown                        NaN
Selected
49996                      NaN                        NaN
Rejected
49997                  pending                    Flagged             On
Hold
49998                  pending                    Flagged
Selected
49999                  Pending                    pending
REJECTED

      rejection_reason       onboarding_date  is_currently_active
49990              NaN             4/7/2021                    1
49991              NaN           03.05.2022                FALSE
49992              NaN                  NaN                   no
49993              NaN            2-Dec-21                    0
49994              NaN                  NaN                   No
49995              NaN  2021-02-18T00:00:00Z                    Y
49996              NaN                  NaN                FALSE
49997              NaN                  NaN                FALSE
49998              NaN               Dec-22                   NO
49999              NaN                    -                 TRUE

[10 rows x 33 columns]
```

```
# See the data types of each column
print("=== DATA TYPES ===")
print(df.dtypes)
```

```
=== DATA TYPES ===
driver_id                    object
driver_name                  object
age                          object
city                         object
phone                        object
email                        object
platform                     object
vehicle_type                 object
vehicle_condition            object
experience_level             object
preferred_shift              object
reason_for_joining           object
languages_known              object
overall_driver_rating        object
total_trips_completed       float64
```

```
trip_completion_rate_%        float64
cancellation_rate_%           float64
avg_daily_earnings_INR         object
total_complaints              float64
total_compliments             float64
peak_hour_availability         object
interview_date                 object
communication_score_10        float64
driving_test_score_10         float64
route_knowledge_score_10      float64
customer_handling_score_10    float64
overall_interview_score_10    float64
document_verification          object
background_check_status        object
interview_result               object
rejection_reason               object
onboarding_date                object
is_currently_active            object
dtype: object
```

```
# Count missing values in every column
print("=== MISSING VALUES PER COLUMN ===")
missing = df.isnull().sum()
missing_pct = (missing / len(df) * 100).round(1)
missing_report = pd.DataFrame({
    'Missing Count': missing,
    'Missing %': missing_pct
})
print(missing_report[missing_report['Missing Count'] > 0].to_string())
```

```
=== MISSING VALUES PER COLUMN ===
                           Missing Count  Missing %
driver_id                            995        2.0
age                                 2457        4.9
phone                               3911        7.8
email                               4605        9.2
platform                            2239        4.5
vehicle_type                        2225        4.4
vehicle_condition                  10532       21.1
experience_level                    2224        4.4
preferred_shift                    11227       22.5
reason_for_joining                 11363       22.7
languages_known                     2153        4.3
overall_driver_rating               3967        7.9
total_trips_completed               4070        8.1
trip_completion_rate_%              4435        8.9
cancellation_rate_%                 4516        9.0
avg_daily_earnings_INR              2511        5.0
total_complaints                    4035        8.1
total_compliments                   3976        8.0
```

```
interview_date                    4933        9.9
communication_score_10            4584        9.2
driving_test_score_10             4441        8.9
route_knowledge_score_10          4552        9.1
customer_handling_score_10        4500        9.0
overall_interview_score_10        4455        8.9
document_verification            13767       27.5
background_check_status          13451       26.9
interview_result                  4757        9.5
rejection_reason                 37826       75.7
onboarding_date                  23828       47.7
```

```
# Step 4 — Replace Fake Empty Values with Real NaN

""" What is the problem?
Our dataset has many ways to say "I don't know" or "no value":
- A dash: `"-"`
- The word: `"na"`, `"none"`, `"unknown"`, `"null"`
- Literally empty: `""`
"""

# Define all the "fake empty" values we want to replace
SENTINEL_VALUES = {
    '-', 'na', 'NA', 'none', 'None', 'NONE',
    'unknown', 'Unknown', 'UNKNOWN',
    'null', 'NULL', 'n/a', 'N/A', ''
}


# This function checks a single value and returns NaN if it's a fake
empty
def replace_sentinel(value):
    if pd.isna(value):          # Already a real NaN? Keep it
        return np.nan
    cleaned = str(value).strip()  # Remove spaces around the value
    if cleaned in SENTINEL_VALUES:
        return np.nan           # Replace fake empty with real NaN
    return value                # Otherwise keep the original value

# Apply this function to every cell in every text column
for col in df.columns:
    if df[col].dtype == object or str(df[col].dtype) == 'string':
        df[col] = df[col].apply(replace_sentinel)

print(f"Total missing values now: {df.isnull().sum().sum():,}")
```

```
Total missing values now: 236,757
```

```
# Step 5 — Drop Rows with No Driver ID
"""
What is the problem?**
```

```
The `driver_id` column is our **Primary Key** — it's the unique identifier for each driver.
If a row has no `driver_id`, we have no way to know who this record belongs to.
Such rows are useless for analysis and machine learning, so we remove them.
"""
# Count rows before dropping
rows_before = len(df)
print(rows_before)
```

```
50000
```

```
# Drop any rows where driver_id is missing (NaN)
df = df.dropna(subset=['driver_id'])

# Count rows after dropping
rows_after = len(df)
print(rows_after)
```

```
49005
```

```
# Step 6 — Fix the Platform Column

""" What is the problem?
The `platform` column should only have a few valid values: `uber`, `rapido`, `ola`, `uber & rapido`
"""


print("Unique platform values BEFORE cleaning:")
print(df['platform'].value_counts(dropna=False))
```

```
Unique platform values BEFORE cleaning:
platform
UBER             4580
ola              4551
Uber & Rapido    4537
rapido           4529
uber             4522
RAPIDO           4495
Uber             4483
Rapido           4441
Uber             4409
Rapido           4397
NaN              3938
rapido             48
uber & rapido      43
uber               32
Name: count, dtype: int64
```

```
# Function to clean and standardise the platform column
def clean_platform(value):
    if pd.isna(value):
        return np.nan               # Keep missing as NaN

    v = str(value).strip().lower()  # Remove spaces, make all
lowercase

    # If both "uber" and "rapido" are in the value, it's a combo
platform
    if 'rapido' in v and 'uber' in v:
        return 'uber & rapido'

    # Individual platforms
    if 'rapido' in v:
        return 'rapido'
    if 'uber' in v:
        return 'uber'
    if 'ola' in v:
        return 'ola'

    return np.nan  # Anything else we don't recognise → missing

# Apply the cleaning function
df['platform'] = df['platform'].apply(clean_platform)
print(df['platform'].value_counts(dropna=False))
```

```
platform
uber             18026
rapido           17910
uber & rapido     4580
ola               4551
NaN               3938
Name: count, dtype: int64
```

```
#Fix City Names (Types)
print(sorted(df['city'].dropna().unique()))
```

```
['AHMEDABAD', 'Ahmedabad', 'BANGALORE', 'Bangalore', 'Bangalroe',
'CHENNAI', 'Chenna', 'Chennai', 'DELHI', 'Delhi', 'Dlhi', 'HYDERABAD',
'Hyderabad', 'Hydrabad', 'JAIPUR', 'Jaipur', 'KOLKATA', 'Kolkata',
'LUCKNOW', 'Lucknow', 'MUMBAI', 'Mumabi', 'Mumbai', 'PUNE', 'Pune',
'ahmedabad', 'bangalore', 'bangalroe', 'chenna', 'chennai', 'delhi',
'dlhi', 'hyderabad', 'hydrabad', 'jaipur', 'kolkata', 'lucknow',
'mumabi', 'mumbai', 'pune']
```

```
# Mapping dictionary: wrong value → correct value
CITY_CORRECTIONS = {
    # Kolkata
    'kolkata': 'Kolkata', 'KOLKATA': 'Kolkata',
    # Chennai
```

```
    'chennai': 'Chennai', 'CHENNAI': 'Chennai',
    'chenna':  'Chennai', 'Chenna':  'Chennai',   # Typo
    # Ahmedabad
    'ahmedabad': 'Ahmedabad', 'AHMEDABAD': 'Ahmedabad',
    # Lucknow
    'lucknow': 'Lucknow', 'LUCKNOW': 'Lucknow',
    # Pune
    'pune': 'Pune', 'PUNE': 'Pune',
    # Delhi
    'delhi': 'Delhi', 'DELHI': 'Delhi',
    'dlhi':  'Delhi', 'Dlhi':  'Delhi',           # Typo
    # Mumbai
    'mumbai': 'Mumbai', 'MUMBAI': 'Mumbai',
    'mumabi': 'Mumbai', 'Mumabi': 'Mumbai',        # Typo
    # Hyderabad
    'hyderabad': 'Hyderabad', 'HYDERABAD': 'Hyderabad',
    'hydrabad':  'Hyderabad', 'Hydrabad':  'Hyderabad', # Typo
    # Bangalore
    'bangalore':  'Bangalore', 'BANGALORE': 'Bangalore',
    'bangalroe':  'Bangalore', 'Bangalroe': 'Bangalore', # Typo
    # Jaipur
    'jaipur': 'Jaipur', 'JAIPUR': 'Jaipur',
}

# Apply the corrections using pandas .replace()
df['city'] = df['city'].replace(CITY_CORRECTIONS)
print(sorted(df['city'].dropna().unique()))
```

```
['Ahmedabad', 'Bangalore', 'Chennai', 'Delhi', 'Hyderabad', 'Jaipur',
'Kolkata', 'Lucknow', 'Mumbai', 'Pune']
```

```
print(f"\nCity distribution:")
print(df['city'].value_counts())
```

```
City distribution:
city
Lucknow      5024
Bangalore    4992
Mumbai       4968
Pune         4942
Delhi        4927
Ahmedabad    4867
Hyderabad    4854
Jaipur       4843
Kolkata      4830
Chennai      4758
Name: count, dtype: int64
```

```python
#Convert Age column to Numeric

df['age'] = pd.to_numeric(df['age'], errors='coerce')

#fix The Age Column
print("Top 20 age values BEFORE cleaning:")
print(df['age'].value_counts(dropna=False).head(20))
```

```
Top 20 age values BEFORE cleaning:
age
NaN      5425
999.0    1458
46.0     1166
26.0     1163
43.0     1157
24.0     1155
23.0     1137
37.0     1134
36.0     1128
47.0     1126
48.0     1122
35.0     1119
51.0     1118
54.0     1116
50.0     1114
39.0     1114
42.0     1114
32.0     1111
41.0     1110
22.0     1105
Name: count, dtype: int64
```

```python
# Remove Impossible Ages

invalid_age_mask = ((df['age'] < 18) | (df['age'] > 70)) &
df['age'].notna()

invalid_count = invalid_age_mask.sum()
df.loc[invalid_age_mask, 'age'] = np.nan

print(f"Invalid ages removed: {invalid_count:,}")
print(df['age'].describe().round(1))
```

```
Invalid ages removed: 3,894
count    39686.0
mean        37.5
std         10.4
min         20.0
25%         29.0
50%         38.0
75%         46.0
```

```
max         55.0
Name: age, dtype: float64
```

```
# Fix Boolean Column
"""
These should simply be **True or False**, but the data has **12
different versions**:
`TRUE`, `FALSE`, `Yes`, `No`, `yes`, `no`, `YES`, `NO`, `Y`, `N`, `1`,
`0`

We will standardise all of these to:
- **1** = Yes / True / Active
- **0** = No / False / Inactive

"""
print("is_currently_active values BEFORE cleaning:")
print(df['is_currently_active'].value_counts(dropna=False))
```

```
is_currently_active values BEFORE cleaning:
is_currently_active
TRUE     7119
FALSE    7033
N        3606
0        3515
Yes      3515
yes      3510
1        3473
Y        3463
No       3459
YES      3451
NO       3439
no       3422
Name: count, dtype: int64
```

```
# All the values that mean "YES/TRUE"
TRUE_VALUES  = {'true', 'yes', '1', 'y'}

# All the values that mean "NO/FALSE"
FALSE_VALUES = {'false', 'no', '0', 'n'}

def standardise_boolean(value):
    if pd.isna(value):
        return np.nan                    # Missing stays missing

    v = str(value).strip().lower()       # Make lowercase and remove
spaces

    if v in TRUE_VALUES:
        return 1                         # → 1 (Yes/True)
    if v in FALSE_VALUES:
```

```
        return 0                                # → 0 (No/False)

    return np.nan                               # Anything else → missing


# Apply to both boolean columns
for col in ['is_currently_active', 'peak_hour_availability']:
    df[col] = df[col].apply(standardise_boolean)
    print(f"{col} cleaned:")
    print(f"{df[col].value_counts(dropna=False).to_dict()}\n")
```

```
is_currently_active cleaned:
{1: 24531, 0: 24474}

peak_hour_availability cleaned:
{1: 24770, 0: 24235}
```

```
#Fix All Catogorical text Column

VEHICLE_TYPE_MAP = {
    'sedan': 'Sedan', 'Sedan': 'Sedan', 'SEDAN': 'Sedan',
    'mini':  'Mini',  'Mini':  'Mini',  'MINI':  'Mini',
    'suv':   'SUV',   'Suv':   'SUV',   'SUV':   'SUV',
    'auto':  'Auto',  'Auto':  'Auto',  'AUTO':  'Auto',
    'bike':  'Bike',  'Bike':  'Bike',  'BIKE':  'Bike',
}

df['vehicle_type'] = df['vehicle_type'].apply(
    lambda v: VEHICLE_TYPE_MAP.get(str(v).strip(), np.nan) if
pd.notna(v) else np.nan
)
print("vehicle_type cleaned:",
df['vehicle_type'].value_counts().to_dict())
```

```
vehicle_type cleaned: {'Mini': 9080, 'SUV': 9002, 'Bike': 8996,
'Sedan': 8990, 'Auto': 8916}
```

```
# Now for Vihicle Condition
VEHICLE_CONDITION_MAP = {
    'good':      'Good',      'Good':      'Good',      'GOOD':
'Good',
    'excellent': 'Excellent', 'Excellent': 'Excellent', 'EXCELLENT':
'Excellent',
    'poor':      'Poor',      'Poor':      'Poor',      'POOR':
'Poor',
    'average':   'Average',   'Average':   'Average',   'AVERAGE':
'Average',
    'avg':       'Average',   'Avg':       'Average',   # Short form →
full word
}
```

```
df['vehicle_condition'] = df['vehicle_condition'].apply(
    lambda v: VEHICLE_CONDITION_MAP.get(str(v).strip(), np.nan) if
pd.notna(v) else np.nan
)
print("□ vehicle_condition cleaned:",
df['vehicle_condition'].value_counts().to_dict())
```

```
□ vehicle_condition cleaned: {'Good': 12387, 'Excellent': 8212,
'Average': 8201, 'Poor': 8175}
```

```
# Shift Map

SHIFT_MAP = {
    'day':   'Day',   'Day':   'Day',   'DAY':   'Day',
    'night': 'Night', 'Night': 'Night', 'NIGHT': 'Night',
    'both':  'Both',  'Both':  'Both',  'BOTH':  'Both',
}
df['preferred_shift'] = df['preferred_shift'].apply(
    lambda v: SHIFT_MAP.get(str(v).strip(), np.nan) if pd.notna(v)
else np.nan
)
print("□ preferred_shift cleaned:",
df['preferred_shift'].value_counts().to_dict())
```

```
□ preferred_shift cleaned: {'Night': 13582, 'Day': 13476, 'Both':
9153}
```

```
# Experience Mapping

EXPERIENCE_MAP = {
    'fresher': 'Fresher', 'Fresher': 'Fresher', 'FRESHER': 'Fresher',
    '0-1 yr':  '0-1 yr',  '0-1 Yr':  '0-1 yr',
    '1-3 yrs': '1-3 yrs', '1-3 Yrs': '1-3 yrs',
    '3-5 yrs': '3-5 yrs', '3-5 Yrs': '3-5 yrs',
    '5+ yrs':  '5+ yrs',  '5+ Yrs':  '5+ yrs',
}
df['experience_level'] = df['experience_level'].apply(
    lambda v: EXPERIENCE_MAP.get(str(v).strip(), np.nan) if
pd.notna(v) else np.nan
)
print("□ experience_level cleaned:",
df['experience_level'].value_counts().to_dict())
```

```
□ experience_level cleaned: {'0-1 yr': 9184, 'Fresher': 9081, '3-5
yrs': 8982, '5+ yrs': 8920, '1-3 yrs': 8857}
```

```
# Result

RESULT_MAP = {
    'selected': 'Selected', 'Selected': 'Selected', 'SELECTED':
'Selected',
```

```
    'rejected': 'Rejected', 'Rejected': 'Rejected', 'REJECTED':
'Rejected',
    'on hold':  'On Hold',  'On Hold':  'On Hold',
    'waitlist': 'Waitlist', 'Waitlist': 'Waitlist',
}
df['interview_result'] = df['interview_result'].apply(
    lambda v: RESULT_MAP.get(str(v).strip(), np.nan) if pd.notna(v)
else np.nan
)
print("□ interview_result cleaned:",
df['interview_result'].value_counts().to_dict())
```

```
□ interview_result cleaned: {'Selected': 16745, 'Rejected': 12988, 'On
Hold': 11590, 'Waitlist': 1246}
```

```
# Document Varification

DOC_MAP = {
    'pass':    'Pass',    'Pass':    'Pass',    'PASS':    'Pass',
    'fail':    'Fail',    'Fail':    'Fail',    'FAIL':    'Fail',
    'pending': 'Pending', 'Pending': 'Pending', 'PENDING': 'Pending',
}
df['document_verification'] = df['document_verification'].apply(
    lambda v: DOC_MAP.get(str(v).strip(), np.nan) if pd.notna(v) else
np.nan
)
print("□ document_verification cleaned:",
df['document_verification'].value_counts().to_dict())
```

```
□ document_verification cleaned: {'Fail': 11308, 'Pending': 11245,
'Pass': 11127}
```

```
# BackGround Check Status

BG_MAP = {
    'clear':   'Clear',   'Clear':   'Clear',   'CLEAR':   'Clear',
    'flagged': 'Flagged', 'Flagged': 'Flagged', 'FLAGGED': 'Flagged',
    'pending': 'Pending', 'Pending': 'Pending', 'PENDING': 'Pending',
}
df['background_check_status'] = df['background_check_status'].apply(
    lambda v: BG_MAP.get(str(v).strip(), np.nan) if pd.notna(v) else
np.nan
)
print("□ background_check_status cleaned:",
df['background_check_status'].value_counts().to_dict())
```

```
□ background_check_status cleaned: {'Clear': 11388, 'Pending': 11333,
'Flagged': 11287}
```

```
# Fix Numerical Columns ( Remove Impossible Values )
```

```
numeric_cols = [
    'total_trips_completed', 'trip_completion_rate_%',
'cancellation_rate_%',
    'avg_daily_earnings_INR', 'total_complaints', 'total_compliments',
    'overall_driver_rating', 'communication_score_10',
'driving_test_score_10',
    'route_knowledge_score_10', 'customer_handling_score_10',
'overall_interview_score_10'
]

for col in numeric_cols:
    df[col] = pd.to_numeric(df[col], errors='coerce')
    # errors='coerce' → if value can't be a number, make it NaN

print("□ All numeric columns converted to proper numbers")
```

```
□ All numeric columns converted to proper numbers
```

```
# Define valid ranges for each column
# Use None if there is no lower or upper limit
VALID_RANGES = {
    'trip_completion_rate_%':    (0, 100),   # A percentage must be 0
to 100
    'cancellation_rate_%':       (0, 100),   # A percentage must be 0
to 100
    'total_trips_completed':     (0, None),  # Can't have negative
trips
    'avg_daily_earnings_INR':    (0, None),  # Can't earn negative
money
    'overall_driver_rating':     (1, 5),     # Rating scale is 1 to 5
    'communication_score_10':    (0, 10),    # Score out of 10
    'driving_test_score_10':     (0, 10),    # Score out of 10
    'route_knowledge_score_10':  (0, 10),    # Score out of 10
    'customer_handling_score_10':(0, 10),    # Score out of 10
    'overall_interview_score_10':(0, 10),    # Score out of 10
}


# Apply the range constraints
print("Removing out-of-range values:\n")
for col, (min_val, max_val) in VALID_RANGES.items():
    # Build a mask (True = invalid row)
    invalid_mask = pd.Series([False] * len(df), index=df.index)

    if min_val is not None:
        invalid_mask = invalid_mask | (df[col] < min_val)  # Below
minimum
    if max_val is not None:
        invalid_mask = invalid_mask | (df[col] > max_val)  # Above
maximum
```

```
    # Only flag rows that actually have a value (not already NaN)
    invalid_mask = invalid_mask & df[col].notna()

    # Replace invalid values with NaN
    df.loc[invalid_mask, col] = np.nan

    print(f"  □ {col}: {invalid_mask.sum():,} invalid values → NaN")
```

```
Removing out-of-range values:

  □ trip_completion_rate_%: 4,908 invalid values → NaN
  □ cancellation_rate_%: 5,042 invalid values → NaN
  □ total_trips_completed: 2,365 invalid values → NaN
  □ avg_daily_earnings_INR: 970 invalid values → NaN
  □ overall_driver_rating: 4,961 invalid values → NaN
  □ communication_score_10: 4,821 invalid values → NaN
  □ driving_test_score_10: 4,923 invalid values → NaN
  □ route_knowledge_score_10: 4,830 invalid values → NaN
  □ customer_handling_score_10: 4,895 invalid values → NaN
  □ overall_interview_score_10: 4,823 invalid values → NaN
```

```
#Parse Date Columns

# Function to try multiple date formats and return a proper datetime
def parse_date_flexible(value):
    if pd.isna(value):
        return pd.NaT              # NaT = "Not a Time" (like NaN but
for dates)

    v = str(value).strip()

    # Try each date format in order
    formats_to_try = [
        '%d-%b-%y',   # e.g. 10-Jul-23
        '%b-%y',      # e.g. Jun-21
        '%m/%d/%Y',   # e.g. 3/5/2021
        '%d/%m/%Y',   # e.g. 27/5/2021
        '%Y-%m-%d',   # e.g. 2021-05-27
    ]

    for fmt in formats_to_try:
        try:
            return pd.to_datetime(v, format=fmt)   # Try this format
        except:
            continue                                # If it fails, try
next

    # If none of the formats worked, try pandas auto-detection
    try:
```

```
        return pd.to_datetime(v, dayfirst=False)
    except:
        return pd.NaT              # If nothing works, return missing

# Apply to both date columns
df['interview_date']  =
df['interview_date'].apply(parse_date_flexible)
df['onboarding_date'] =
df['onboarding_date'].apply(parse_date_flexible)

print(f"   interview_date  — valid dates:
{df['interview_date'].notna().sum():,}")
print(f"   onboarding_date — valid dates:
{df['onboarding_date'].notna().sum():,}")
```

```
   interview_date  — valid dates: 44,181
   onboarding_date — valid dates: 18,576
```

```
# Show a sample of parsed dates
print("\nSample of interview dates after parsing:")
print(df['interview_date'].dropna().head(5))
```

```
Sample of interview dates after parsing:
0    2023-07-10 00:00:00
1    2021-06-01 00:00:00
2    2021-02-01 00:00:00
3    2021-03-05 00:00:00
5    2023-03-01 00:00:00
Name: interview_date, dtype: object
```

```
# Remove Duplicate Rows

# Count before
rows_before = len(df)

# Drop duplicate driver_ids — keep the first occurrence
df = df.drop_duplicates(subset=['driver_id'], keep='first')

rows_after = len(df)
print(f"   Rows before : {rows_before:,}")
print(f"   Duplicates  : {rows_before - rows_after:,}")
print(f"   Rows after  : {rows_after:,}")
```

```
   Rows before : 49,005
   Duplicates  : 2,752
   Rows after  : 46,253
```

```
#Final Check and Save the Clean Dataset

print(f"   Total rows    : {len(df):,}")
```

```python
print(f"   Total columns : {df.shape[1]}")
print("Remaining missing values per column:")
missing_final = df.isnull().sum()
missing_pct   = (missing_final / len(df) * 100).round(1)
summary = pd.DataFrame({
    'Missing Count': missing_final,
    'Missing %':     missing_pct
})
print(summary.to_string())
```

```
   Total rows    : 46,253
   Total columns : 33
Remaining missing values per column:
                           Missing Count  Missing %
driver_id                              0        0.0
driver_name                            0        0.0
age                                 8811       19.0
city                                   0        0.0
phone                               3625        7.8
email                               4258        9.2
platform                            3739        8.1
vehicle_type                        3808        8.2
vehicle_condition                  11389       24.6
experience_level                    3767        8.1
preferred_shift                    12129       26.2
reason_for_joining                 16420       35.5
languages_known                     3744        8.1
overall_driver_rating               9295       20.1
total_trips_completed               6016       13.0
trip_completion_rate_%              8745       18.9
cancellation_rate_%                 8901       19.2
avg_daily_earnings_INR             10630       23.0
total_complaints                    3780        8.2
total_compliments                   3702        8.0
peak_hour_availability                 0        0.0
interview_date                      4558        9.9
communication_score_10              8772       19.0
driving_test_score_10               8791       19.0
route_knowledge_score_10            8781       19.0
customer_handling_score_10          8807       19.0
overall_interview_score_10          8637       18.7
document_verification              14480       31.3
background_check_status            14185       30.7
interview_result                    6052       13.1
rejection_reason                   37840       81.8
onboarding_date                    28703       62.1
is_currently_active                    0        0.0
```

```python
# Preview The Clean Data
print("\\nFirst 5 rows of the CLEAN dataset:")
```

```
df[['driver_id', 'driver_name', 'age', 'city', 'platform',
    'vehicle_type', 'experience_level', 'overall_driver_rating',
    'interview_result', 'is_currently_active']].head()

\nFirst 5 rows of the CLEAN dataset:

   driver_id       driver_name   age       city platform vehicle_type
\
0  DRV016649    Naresh Chauhan   NaN    Chennai     uber         Auto

1  DRV032785        Amit Gupta  45.0  Ahmedabad     uber         Mini

2  DRV011323     Yogesh Nair    53.0    Lucknow     uber          SUV

3  DRV022168       Manoj Patel  46.0  Hyderabad   rapido         Mini

4  DRV040782  Vikram Chaudhary  25.0    Kolkata     uber          SUV


  experience_level  overall_driver_rating interview_result  \
0           0-1 yr                    NaN          On Hold
1          Fresher                    NaN          On Hold
2           0-1 yr                    NaN         Selected
3          1-3 yrs                    4.8         Selected
4          Fresher                    4.4         Rejected

   is_currently_active
0                    0
1                    0
2                    0
3                    0
4                    0

# Save The Clean Dataset

output_path = 'uber_rapido_driver_CLEAN.csv'
df.to_csv(output_path, index=False)

print(f"Clean dataset saved as: {output_path}")
print(f"This file is now ready for Phase 3 (EDA) and Phase 5 (ML
Model)")

Clean dataset saved as: uber_rapido_driver_CLEAN.csv
This file is now ready for Phase 3 (EDA) and Phase 5 (ML Model)
```